

1-4 操作系统结构



1. 本节主线

- 内容总览
 - 模块化结构：按功能把 OS 划分成多个模块。
 - 分层式结构：把 OS 划分为若干层，每层调用低层服务。
 - 微内核结构：只把最基本功能留在内核，大部分 OS 功能放到用户态服务器中。
 - 外核结构：尽量缩小内核，把更多资源管理权交给应用程序。
- 核心问题
 - 如何组织 OS 功能：按模块、按层次，或把功能移出内核。
 - 如何兼顾可维护性和效率：结构越清晰通常越好维护，但可能带来额外开销。
 - 如何控制内核大小：微内核和外核都追求更小的内核，但侧重点不同。

2. 模块化结构

- 基本概念
 - 定义：将操作系统按照功能划分成若干模块。
 - 模块职责：每个模块具有某一方面的管理功能。
 - 交互方式：各模块之间通过接口交互。
 - 例子：进程管理、存储器管理、文件管理等。
- 模块内部细分
 - 进程管理：进程调度等。
 - 存储器管理：内存分配等。
 - 文件管理：目录管理等。
 - 特点：大功能可继续划分为更小的子功能。
- 衡量模块独立性的标准
 - 内聚性：模块内各组成部分之间联系的密切程度。
 - 内聚性越高：模块内部越统一，模块独立性越好。
 - 耦合度：模块之间相互联系和影响的程度。
 - 耦合度越低：模块之间依赖越少，模块独立性越好。
- 优势
 - 提升设计质量：提高 OS 的正确性、可理解性和可维护性。
 - 增强适应能力：使 OS 更容易适应需求和环境变化。
 - 加快开发进度：多个模块可相对并行开发，缩短开发周期。
- 劣势
 - 接口难以预设：设计初期规定的模块接口，可能无法满足后续真实需求。
 - 决策缺乏顺序：模块化要求多个模块同步推进，设计决策之间缺少可靠的先后依赖关系。
 - 协调成本较高：模块之间接口一旦变化，可能影响其他模块。

3. 分层式结构

- 基本概念
 - 定义：将操作系统分为若干层。
 - 调用规则：每层只能调用比它更低层的功能和服务。
 - 底层基础：最底层接近裸机。
 - 上层功能：逐层向上提供更高级的 OS 服务。
- 层次关系
 - 裸机：最底层硬件基础。
 - 内核基础功能：原语、中断处理、时钟管理等。
 - 资源管理层：进程管理、存储器管理、设备管理等。
 - 非内核功能：更高层的 OS 服务和用户接口。
- 优点
 - 便于系统调试：层次清楚，容易定位问题。
 - 易于扩充：可在已有层次基础上增加功能。
 - 易于维护：每层职责相对明确。
 - 结构清晰：有利于理解 OS 的组织方式。
- 缺点
 - 合理定义各层困难：功能边界不一定容易划分。
 - 不够灵活：每层只能调用低层，跨层调用受限。
 - 效率较差：层层调用会带来额外开销。

4. 微内核结构

- 基本概念
 - 定义：仅将内核中最基本的功能保留在内核。
 - 外移思想：OS 的绝大部分功能放在微内核外面的一组服务器中实现。
 - 服务器位置：这些服务器作为进程运行在用户态。
 - 通信方式：用户进程、微内核、服务器之间通过消息传递交互。
- 微内核中保留的功能
 - 最基本机制：如低级地址空间管理、线程管理、进程间通信等。
 - 核心原则：能放到用户态的服务尽量不放进内核。
 - 目标：减小内核规模，提高系统可靠性和可扩展性。
- 请求处理过程
 - 发起请求：应用程序向微内核发送消息请求，如读取文件。
 - 消息传递：微内核把请求转发给用户空间中的文件系统服务器。
 - 处理请求：文件系统服务器解析请求，访问磁盘并读取文件内容。
 - 返回结果：服务器把结果经微内核返回给应用程序。
 - 继续执行：应用程序收到数据后继续运行。
- 优势
 - 提高系统可扩展性：新增服务可作为用户态服务器加入。
 - 增强系统可靠性：某个服务器出错不一定导致整个内核崩溃。
 - 增强可移植性：内核小，与硬件相关部分更集中。
 - 支持分布式系统：消息传递机制天然适合跨机器服务交互。
 - 融入面向对象技术：服务可封装成相对独立的对象或组件。
- 劣势
 - 运行效率低：频繁消息传递和用户态/内核态切换会带来开销。
 - 调用链更长：一次服务请求可能经过应用、微内核、服务器、微内核、应用。
 - 实现复杂：需要可靠高效的通信和服务管理机制。

5. 外核结构

- 基本概念
 - 定义：对资源进行划分，给每个用户或应用分配一部分资源。
 - 核心目标：让应用程序可以直接请求特定物理资源。
 - 请求对象：特定物理空间、特定磁盘块等。
 - 系统职责：只保证被请求的资源当前空闲，并保证安全使用。
- 外核的设计理念
 - 最小化内核：内核尽可能小。
 - 转移管理权：尽可能多地把管理和控制权转移给应用程序。
 - 直接硬件交互：应用程序可以更直接地和硬件资源交互。
 - 安全保护：外核仍负责确保系统安全性和稳定性，防止资源被滥用。
- 外核中的内核职责
 - 保护系统资源：防止应用非法访问或破坏其他应用资源。
 - 资源分配：确认资源是否空闲并分配给应用。
 - 安全策略：通过策略保护硬件资源不被滥用。
 - 最小管理：硬件资源的具体管理职责更多委托给应用程序。
- 优势
 - 应用控制力强：应用可根据自身需要管理资源。
 - 潜在效率高：减少传统内核抽象层带来的开销。
 - 灵活性强：不同应用可采用不同资源管理策略。
- 劣势
 - 应用开发复杂：应用需要承担更多资源管理责任。
 - 安全设计难：必须严格保证资源隔离和保护。
 - 通用性挑战：不同应用直接管理资源，系统统一抽象较弱。

6. 四种结构对比

横向对比

结构	核心思想	优点	缺点
模块化结构	按功能划分模块	易理解、易维护、易扩展	接口难预设，模块协调困难
分层式结构	OS 分层，上层调用下层	易调试、易扩充、结构清晰	分层困难，不灵活，效率较差
微内核结构	最基本功能留内核，服务放用户态	可靠、可扩展、可移植、支持分布式	消息传递和状态切换多，效率低
外核结构	内核只保护和分配资源，应用直接管理资源	应用灵活，潜在效率高	应用复杂，安全保护难